From Free Theorems to Embedded Computations

Lecture 12

Section 1

Recap

Where we are

System F adds quantification over types to STLC:

- New type former: $\forall\alpha.\, A$
- New terms: $\Lambda\alpha.\, t$ (type abstraction) and $t\,[\![A]\!]$ (type application)
- New reduction: $(\Lambda\alpha.\, t)\,[\![A]\!] \to_\beta t[\alpha := A]$

System	**Computes**	**Corresponds to**
STLC	extended polynomials	prop. intuitionistic logic
System T	provably total functions of PA	Gödel's Dialectica
System F	provably total functions of PA_2	2nd-order intuitionistic logic

Section 2

Part I: Parametricity and Free Theorems

The central question

Consider the type $\forall\alpha.\ \alpha \to \alpha$ in System F.

What closed terms can inhabit it?

The central question

Consider the type $\forall\alpha.\ \alpha \to \alpha$ in System F.

What closed terms can inhabit it?

The type variable α is completely **unknown**. Given a value $x : \alpha$, you cannot:

- inspect it (no pattern matching on an unknown type)
- construct a new one (you have no constructors)
- ignore it and return something else (you have nothing else of type α)

The only thing you can do is **return** x **itself**. The type forces $\Lambda\alpha.\ \lambda x{:}\alpha.\ x$.

Slogan

The more polymorphic a type, the more it *constrains* behavior.

A gallery of free theorems

Work through each type: what functions can possibly inhabit it?

Type	**What it forces**
$\forall\alpha.\ \alpha \to \alpha$	must be id
$\forall\alpha.\ \alpha \to \alpha \to \alpha$	pick first or second (Church booleans)
$\forall\alpha.\ [\alpha] \to [\alpha]$	rearrange, drop, or duplicate — never inspec
$\forall\alpha\,\beta.\ (\alpha \to \beta) \to [\alpha] \to [\beta]$	apply f to every element retained (naturality
$\forall\alpha.\ (\alpha \to \alpha) \to \alpha \to \alpha$	apply f exactly n times (Church numerals)

These are called **free theorems** (Wadler 1989): they are *free* because you did no work to prove them — they follow from the type signature alone, without reading a single line of implementation.

Free theorem: $\forall\alpha.\ \alpha \to \alpha \to \alpha$

Given $x, y : \alpha$ for an unknown α, what can you return?

Free theorem: $\forall\alpha.\ \alpha \to \alpha \to \alpha$

Given $x, y : \alpha$ for an unknown α, what can you return?

You cannot inspect x or y (unknown type), cannot construct anything new (no constructors for α), and must return something of type α. The only values of type α in scope are x and y.

So the function must be either $\Lambda\alpha.\,\lambda x.\,\lambda y.\,x$; or ; $\Lambda\alpha.\,\lambda x.\,\lambda y.\,y$.

There are exactly **two** inhabitants. These are the Church booleans: true picks the first, false picks the second. The type *is* the data type — no constructors were declared; they emerged from the constraint.

Free theorem: $\forall\alpha.\ (\alpha \to \alpha) \to \alpha \to \alpha$

Given $f : \alpha \to \alpha$ and $x : \alpha$, produce a value of type α.

Free theorem: $\forall\alpha.\ (\alpha \to \alpha) \to \alpha \to \alpha$

Given $f : \alpha \to \alpha$ and $x : \alpha$, produce a value of type α.

The only way to build values of type α is to apply f to what you have. Starting from x: ; x, ; $f\,x$, ; $f(f\,x)$, ; $f(f(f\,x))$, ; ...

Every inhabitant is $\lambda f.\,\lambda x.\,f^n\,x$ for some fixed $n \geq 0$.

These are the **Church numerals**:

$$\mathsf{c}_0 = \lambda f.\,\lambda x.\,x, \quad \mathsf{c}_1 = \lambda f.\,\lambda x.\,f\,x, \quad \mathsf{c}_2 = \lambda f.\,\lambda x.\,f(f\,x), \quad \ldots$$

The natural numbers are *forced into existence* by the type. This is the Church encoding from Lecture 11.

Free theorem: the map type

The type $\forall \alpha\, \beta.\ (\alpha \to \beta) \to [\alpha] \to [\beta]$ almost forces the function to be `map`.

α and β are unknown. Given $f : \alpha \to \beta$ and a list of αs, any β in the output must come from applying f. You cannot:

- produce a β from nothing (you have no β constructors)
- ignore f (then you have no βs at all)

The function can still drop or reorder elements, but every output element is f applied to some input. This is a **naturality condition**: a free theorem from the type alone.

Incidentally: what exactly *is* the list constructor in $[\alpha]$? It takes a type and returns a type. That is something new, not yet in System F.

Section 3

Part II: Type Constructors and System Fω

Types that take types as arguments

In System F, types are formed from:

$$A, B \ ::= \ \alpha \ \mid \ A \to B \ \mid \ \forall \alpha.\, A$$

But we want to write List α, Maybe α, Either ε α. These require **type constructors**: functions from types to types.

Constructor	**What it does**
List	takes a type α, gives the type of α-lists
Maybe	takes a type α, gives the type of optional αs
Either	takes two types ε, α, gives their sum

List alone is *not a type*. List Int is. Just as f alone may be ill-typed without an argument, List alone is ill-*kinded* without one.

Kinds: the types of types

We need a new layer to classify type expressions. This is the **kind system**.

$$\kappa \ ::= \ * \ \mid \ \kappa_1 \to \kappa_2$$

- $*$ is the kind of **proper types** — things that classify terms
- $\kappa_1 \to \kappa_2$ is the kind of type constructors

Expression	**Kind**
Int, Bool, Int $\to$ Bool	$*$
List, Maybe	$* \to *$
Either	$* \to * \to *$
Either String	$* \to *$ (partial application)
List Int	$*$ (fully applied)

The kinding rules

We now have two kinds of judgment:

- $\Gamma \vdash t : A$ (term t has type A, as before)
- $\Gamma \vdash A :: \kappa$ (type expression A has kind κ, new)

$$\frac{\alpha :: \kappa \in \Gamma}{\Gamma \vdash \alpha :: \kappa} \qquad \frac{\Gamma, \alpha :: \kappa \vdash A :: \kappa'}{\Gamma \vdash \lambda \alpha{::}\kappa.\, A \;::\; \kappa \to \kappa'}$$

$$\frac{\Gamma \vdash F :: \kappa \to \kappa' \qquad \Gamma \vdash A :: \kappa}{\Gamma \vdash F\, A :: \kappa'}$$

Well-formedness: a term may only have type A if $\Gamma \vdash A :: *$.

Notice: variables, abstraction, application. These rules are exactly **STLC**, with type constructors playing the role of terms and kinds playing the role of types.

Three levels of abstraction

The pattern of abstraction, application, and typing judgment repeats at every level:

Level	**Abstraction**	**Application**	**Judgment**
Terms	$\lambda x{:}A.\, t$	$t\ s$	$\Gamma \vdash t : A$
Types (System F)	$\Lambda\alpha.\, t$	$t\,[\![A]\!]$	$\Gamma \vdash t : A$
Constructors (Fω)	$\lambda\alpha{::}\kappa.\, A$	$F\ A$	$\Gamma \vdash A :: \kappa$

The extended system is **System Fω**. Fω allows quantification over any kind:

$$\frac{\Gamma,\ \alpha :: \kappa \vdash t : A}{\Gamma \vdash \Lambda\alpha{::}\kappa.\, t\ :\ \forall\alpha{::}\kappa.\, A}$$

What kind-polymorphism buys you

In System F we can write a map function for lists:

$$\texttt{mapList} : \forall \alpha\, \beta.\, (\alpha \to \beta) \to \mathsf{List}\ \alpha \to \mathsf{List}\ \beta$$

But this is specific to List. We would need a separate type for Maybe, Tree, etc.

In Fω we can write the type:

$$\texttt{fmap} : \forall F{::}(* \to *).\, \forall \alpha\, \beta.\, (\alpha \to \beta) \to F\ \alpha \to F\ \beta$$

But this type is *too* generic: for an arbitrary unknown F, we know nothing about its structure, so there is no useful function that inhabits it. The type becomes useful only when we **fix** F to a particular constructor like List or Maybe — because then we know the constructors and can write an implementation.

Why the name Fω?

System F allows quantification over types of kind $*$ only. We can generalize:

System	**Quantify over kinds up to...**
$\mathsf{F}_1 =$ System F	$*$
F_2	$*$ and $* \to *$
F_3	$*$, $* \to *$, $(* \to *) \to *$, ...
F_n	kinds of order $< n$
F_ω	kinds of *all* finite orders — no bound

$\mathsf{F}_\omega = \bigcup_n \mathsf{F}_n$. The subscript ω is the ordinal ω, the first limit ordinal. The kind grammar $\kappa ::= * \mid \kappa \to \kappa$ generates kinds of every finite order, and we impose no upper bound.

This has nothing to do with large cardinals — ω simply means: all finite levels.

What System Fω adds to the hierarchy

System	**Type-level**	**Quantification over**
STLC	nothing	—
System F	substitution only	types of kind $*$
System Fω	λ-calculus on types	types of any kind
Dependent types (Lean)	arbitrary terms	types can depend on terms

Haskell's type system is essentially System Fω plus type classes plus some extensions. The theoretical foundation we have been building is exactly what is under the hood.

Section 4

Part III: Type-Level Computation Always Terminates

The type-level λ-calculus is STLC

As we just observed, the kinding rules give us variables, abstraction, application, and β-reduction at the type level:

$$(\lambda\alpha{::}\kappa.\, A)\, B \;\to_\beta\; A[\alpha := B]$$

This is an instance of STLC operating on types rather than terms: the *terms* of this STLC are type constructors, and the *types* are kinds.

In **Lecture 9** we proved that STLC is weakly normalizing (every term has a normal form) and in fact **strongly normalizing** (every reduction sequence terminates, no matter the strategy).

Corollary

Every type-level reduction sequence in System Fω terminates. There are no infinite loops at the type level. Typechecking in Fω is decidable.

Section 5

Part IV: Functors

Lifting functions through type constructors

We saw that Fω lets us write the type $\forall F{::}(* \to *).\ \forall \alpha\, \beta.\ (\alpha \to \beta) \to F\, \alpha \to F\, \beta$, but it has no inhabitant for generic F. In practice, what we do is provide a specific `fmap` for each constructor:

```
mapList  :: (a -> b) -> [a]      -> [b]
mapMaybe :: (a -> b) -> Maybe a  -> Maybe b
mapTree  :: (a -> b) -> Tree a   -> Tree b
```

Each lifts a function $f : \alpha \to \beta$ through a type constructor, transforming contents without changing shape. We call a type constructor $F :: * \to *$ a **Functor** when we have equipped it with such a function

$$\texttt{fmap}_F :: \forall \alpha\, \beta.\ (\alpha \to \beta) \to F\, \alpha \to F\, \beta$$

satisfying the **functor laws**: $\texttt{fmap id} = \texttt{id}$; and ; $\texttt{fmap}\ (f \circ g) = \texttt{fmap}\ f \circ \texttt{fmap}\ g$.

The Functor type class

In Haskell, a **type class** declares an interface — a set of operations that any type constructor may implement. An **instance** provides the implementation for a specific constructor.

```
class Functor f where                -- declaration: any f :: * -> *
  fmap :: (a -> b) -> f a -> f b     -- must provide this operat

instance Functor Maybe where         -- implementation for Maybe
  fmap _ Nothing  = Nothing
  fmap f (Just x) = Just (f x)
```

You can think of the `class` declaration as giving Functor itself a kind: it expects a type constructor $f :: * \to *$ as argument. Writing Functor Maybe is well-kinded; writing Functor Int is not, because Int $:: *$. System Fω is needed to even state this.

The `instance` is where the actual computation lives: it fixes $f =$ Maybe and provides a concrete definition of `fmap`.

Section 6

Part V: Monads — A First Look

The problem: effects in a pure language

This is a quick introduction to monads — we will develop the ideas in depth next time. For now, the goal is to see *why* the concept exists.

A pure language guarantees: a function called twice with the same arguments returns the same result.

But real programs have **effects** — state, failure, multiple outcomes, IO.

The problem: effects in a pure language

This is a quick introduction to monads — we will develop the ideas in depth next time. For now, the goal is to see *why* the concept exists.

A pure language guarantees: a function called twice with the same arguments returns the same result.

But real programs have **effects** — state, failure, multiple outcomes, IO.

How do you model effects without breaking purity?

Moggi's insight (1991)

Do not *perform* the effect. Instead, **describe** it. Model *a computation of type* α *with effect* T as a value of type $T\,\alpha$. The value is a **description**, not an execution.

Descriptions, not actions

Type	**Read as**	**Effect**
State $s\ \alpha$	yields α while reading/writing state s	mutable state
Maybe α	might fail to yield α	partiality
$[\alpha]$	yields one of several possible α values	branching
IO α	when executed by the runtime, yields α	side effects

In every case, $T\,\alpha$ is a **description** of a computation — a recipe, not the result of running it. You can hold a description, pass it around, combine it with other descriptions, and only later choose to execute it.

The Monad type class

```
class Functor m => Monad m where
  return :: a -> m a
  (>>=)  :: m a -> (a -> m b) -> m b
```

- `return a`: the trivial description that immediately yields a, no effect
- `m >>= f`: run m, take its result x, then run $f\ x$

The **monad laws**:

$$\begin{aligned}
\texttt{return}\ a \gg\!= f &= f\ a && \text{(left unit)} \\
m \gg\!= \texttt{return} &= m && \text{(right unit)} \\
(m \gg\!= f) \gg\!= g &= m \gg\!= (\lambda x.\ f\ x \gg\!= g) && \text{(assoc.)}
\end{aligned}$$

These are the **monoid laws** for program composition.

Section 7

Part VI: The State Monad — A Concrete Example

What is a stateful computation?

Forget Haskell for a moment. Think about what *a computation with mutable state* means.

A stateful computation does two things: it **reads** the current state, and it may **write** a new state. When it is done, it produces a result.

So a stateful computation is a **function**:

$$\text{old state} \longmapsto (\text{new state},\ \text{result})$$

$$\text{State}\ s\ \alpha \equiv s \to (s,\ \alpha)$$

This is not running yet — it is a function *waiting* for a state. You can hold it, pass it around, compose it with other such functions, all without executing anything.

The State monad: naming the pattern

We saw that a stateful computation is a function $s \to (s, \alpha)$. In Haskell, we give this pattern a name:

$$\text{State } s\ \alpha \ \equiv\ s \to (s,\ \alpha)$$

A value of type State Int String is a function Int $\to$ (Int, String) waiting to be called. It has not run yet. It *describes* a computation: *given an integer state, do something and return a string.*

To actually execute it, you provide an initial state — in Haskell, this is called `runState`. Until then, the function just sits there.

The State monad: composing without running

Because State $s\ \alpha$ is just data, we can **manipulate computations without running them**:

- A function State $s\ \alpha \to$ State $s\ \alpha$ takes a stateful computation and returns a *new* one — it transforms the description of the computation to be performed (at some point, once actually invoked), not the result
- $\gg\!=$ (*bind*) chains two descriptions into a *single combined* description

Note: **nothing truly stateful is happening here**. We are composing pure functions of the form $s \to (s, a)$. But the *style* looks imperative. We first *design* the computation by composing descriptions; only at the end do we run it.

Example: imperative-style Fibonacci

Using do-notation (syntactic sugar for $\gg=$), we can write a loop that updates state — exactly as you would in an imperative language:

```
fib :: Int -> Int
fib n = snd (runState loop (0, 1))      -- run it
  where
    loop :: State (Int, Int) ()
    loop = forM_ [1..n] $ \_ -> do
      (a, b) <- get                     -- read the pair
      put (b, a + b)                    -- write the next pair
```

`loop` is a description of type State (Int, Int) $()$: a function that, when given an initial pair, steps through Fibonacci updates n times. Nothing runs until `runState` is called with the initial state $(0, 1)$.

IO as an embedded computation

IO α fits the same framework: it is a **description** of a computation that, when executed, yields an α. You can compose, transform, and store IO descriptions without executing them:

```
greet    :: IO ()
greet    = putStrLn "Hello!"

greet3   :: IO ()
greet3   = greet >> greet >> greet     -- no printing yet

actions  :: [IO ()]
actions  = [greet, putStrLn "Bye"]    -- a list of descriptions
```

None of these lines print anything. They build up descriptions. Execution only happens when the runtime runs `main`:

```
main :: IO ()
main = greet3                          -- now it prints
```

Pure monads vs. IO

State, Maybe, and [] are **pure**: they simulate effects by composing ordinary functions. You can *run* them yourself (`runState`, pattern match on `Maybe`, etc.).

IO is different. The *state* is the entire real world, and you do not control what comes back. Within the safe language, there is no `runIO` — only the runtime can execute IO actions, by running `main`.

	Pure monads	**IO**
State	a value you provide	the real world
Run it yourself?	yes (`runState`, etc.)	no (only `main`)
Deterministic?	same input $\Rightarrow$ same output	no

The monad type declares what effects are allowed

The choice of monad determines what a computation is *permitted* to do:

Type	**What the program may do**
State Int α	read/write an integer counter
State (Array Int Int) α	read/write array cells
Maybe α	fail or succeed
IO α	interact with the outside world
α (no monad)	nothing — a plain value

A function typed State Int $()$ can only touch an integer counter. But IO $()$ is very coarse: it permits interaction with the *entire* operating system.

Effect systems refine this: instead of a single IO, one defines more specific monads — a type that permits only file access, or only network operations. The monad in the signature becomes a fine-grained contract.

Moggi's contribution: the problem

Moggi (1991) was working on **denotational semantics** — assigning a mathematical meaning to each program construct. His question: how do you do this for programs with effects?

Before Moggi, each effect required a different mathematical interpretation:

Effect	**Interpret $A \to B$ as**
state S	$A \times S \to B \times S$
exceptions E	$A \to B + E$
both	$A \times S \to (B + E) \times S$

Each combination required its own bespoke construction.

Moggi's contribution: the common shape

Moggi noticed that all these interpretations have the same shape: a pure program has type $A \to B$; its effectful version has type $A \to T(B)$ for some T.

Effect	$T(B)$
state	$S \to S \times B$
exceptions	$B + E$
partiality	$B + 1$

The functor T encodes which effect you are modeling. An effectful program is not a function $A \to B$ but a function $A \to T(B)$.

But this raises an immediate question: how do you *compose* two such programs?

Moggi's contribution: the composition problem

With pure functions, composition is trivial: given $f : A \to B$ and $g : B \to C$, we get $g \circ f : A \to C$.

But if $f : A \to T(B)$ and $g : B \to T(C)$, we are stuck. f outputs a $T(B)$, but g expects a B. The types do not match — we cannot just compose them.

We need two operations:

- $\texttt{return} : A \to T(A)$ — embed a pure value into an effectful computation that does nothing
- $\gg\!= : T(A) \to (A \to T(B)) \to T(B)$ — given an effectful result and a function that expects the raw value, chain them together

With $\gg\!=$, we can compose f and g: run f, bind the result to g.

The categorical picture (sketch)

Programs and types form a category $\mathcal{C}$: objects are types, morphisms $A \to B$ are programs. A pure program is just a morphism in $\mathcal{C}$.

For a given effect T (state, exceptions, ...), effectful programs have type $A \to T(B)$. These form a new category $\mathcal{C}_T$ — the **Kleisli category** — where:

- the objects are the same types
- a morphism $A \to_T B$ is a function $A \to T(B)$ (an effectful program)
- identity is `return`
- composition is $\gg=$

The monad laws are exactly what guarantee that this composition is associative and that `return` is a genuine identity — i.e. that $\mathcal{C}_T$ is actually a category. This is the mathematical content of the `Monad` type class. We will not develop the category theory further in this course.

Example: error handling without monads

Consider looking up a user, then their department, then the department head. Each step can fail. In Go, you check errors manually at every step:

```
func getHead(uid int) (*User, error) {
    user, err := lookupUser(uid)
    if err != nil { return nil, err }
    dept, err := lookupDept(user.DeptID)
    if err != nil { return nil, err }
    head, err := lookupUser(dept.HeadID)
    if err != nil { return nil, err }
    return head, nil
}
```

Three operations, three `if err != nil` blocks. The logic is buried in boilerplate.

Example: error handling with the Maybe monad

In Haskell, Maybe is a monad. Its $\gg=$ propagates failure automatically: if any step returns `Nothing`, the rest is skipped.

```
Nothing >>= f = Nothing        -- failure: skip f
Just x  >>= f = f x            -- success: continue
```

Using $\gg=$ explicitly:

```
getHead uid =  lookupUser uid  >>= \user ->
               lookupDept (deptID user) >>= \dept ->
               lookupUser (headID dept)
```

Example: do-notation

Haskell provides **do-notation** as syntactic sugar for chains of $\gg=$. The same function:

```
getHead :: Int -> Maybe User
getHead uid = do
  user <- lookupUser uid
  dept <- lookupDept (deptID user)
  lookupUser (headID dept)
```

This reads like imperative code: *look up the user; look up their department; look up the department head.* If any step returns `Nothing`, the rest is skipped automatically.

Compare with the ten-line Go version: same logic, no error-checking boilerplate. The plumbing is inside $\gg=$, written once in the Maybe instance, rather than repeated at every call site.

Section 8

Summary

Next lecture

Next time: monads in depth — the full implementations of `return`, `>>=`, `get`, `put` for the State monad, worked examples, and the Turing machine analogy.

After that: type inference. All of this works in Haskell because type inference is tractable — you rarely write these types by hand.

But System F's type inference is undecidable (Wells, 1999): given a term with no annotations, there is no algorithm that always finds its most general type. The fix is **Hindley–Milner**: restrict to prenex polymorphism, allow generalization only at `let`-bindings, and use unification as the engine.